GitHub Copilot

Feature Walkthrough

AI Integration Engineer Demo · R&D Enablement Program

1 Auto-complete | 2 Ask + /explain | 3 Agent Mode | 4 Tests + Fix | 5 Inline Chat | 6 Instructions

Demo project: Python Task Manager · VS Code

1 Inline Auto-Complete from Comment

How it works

1. Type a natural-language comment describing the function.
2. Press Enter to move to the next line.
3. Copilot shows a ghost suggestion in grey.
4. Press Tab to accept the whole suggestion.
5. Use Alt+] to cycle through alternatives.

Tab Accept · Esc Dismiss · Alt+] Next · Ctrl+Enter Open panel

task_manager.py

```
# task_manager.py

# Function that adds a new task with title,
# priority (low/medium/high), and due date

def add_task(title: str,
             priority: str = "medium",
             due_date: Optional[str] = None) -> dict:
    """Add task to store."""
    task = {
        "id": _next_id, "title": title,
        ...
    }
    _tasks.append(task)
    return task
```

← Copilot ghost suggestion appears here in grey

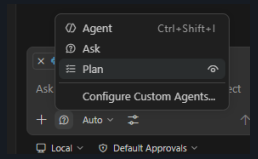
inline completion | comment-driven | Tab to accept

2 Chat Panel — Ask Mode, /explain & Context

Ask Mode

Open Copilot Chat (Ctrl+Alt+I)

Make sure mode = Ask (not Agent)



Type: "What does get_overdue_tasks do?"

Copilot answers using your open file

Does NOT modify files in Ask mode

/explain — Slash Commands

Select a function in the editor

Type /explain in chat → instant explanation

Try /fix on a buggy snippet

Try /tests to generate test stubs

Built-in shortcuts to common tasks

Model & Context

Click model name to switch models

Use #file to add a file as context

Use @workspace to search the repo

Drag a file from Explorer into chat

More context = better answers

```
You  /explain get_overdue_tasks  →  Copilot:  Returns all incomplete tasks whose due_date is before today...
```

3 Agent Mode — Autonomous Multi-Step Coding

What is Agent Mode?

Agent mode lets Copilot autonomously:

- Read and write multiple files
- Run terminal commands
- Iterate until the task is done
- Ask clarifying questions if needed

Agent vs Ask

Ask mode → answers questions, read-only

Agent mode → edits code, runs commands, loops

1. Switch to Agent mode in the chat panel
2. Type: "Add get_tasks_by_priority() and get_overdue_tasks() to task_manager.py"
3. Copilot reads the file, proposes changes, shows diff
4. Review diff → Accept / Discard each change
5. Copilot may run python task_manager.py to self-verify

autonomous agent | multi-file edits | terminal access

4 Agent Mode — Generate Tests & Auto-Fix

Demo Prompt (Agent mode)

"Write pytest tests for task_manager.py. Cover all public functions, include edge cases and fixtures that reset state. Save to test_task_manager.py, run pytest, and fix any failures automatically."

What Agent does next:

1. Reads task_manager.py to understand the API
2. Generates test_task_manager.py with full coverage
3. Runs: pytest test_task_manager.py in terminal
4. If tests fail → reads error, patches code
5. Re-runs pytest → iterates until green ✓

Terminal

```
$ pytest test_task_manager.py -v
FAILED test_task_manager.py::TestCompleteTask::test_completed_at_is_set
AssertionError: 'completed_at' not in task

▶ Copilot analyzing failure...
▶ Patching complete_task() → adding completed_at field

$ pytest test_task_manager.py -v
PASSED  TestAddTask::test_returns_dict_with_expected_keys
PASSED  TestCompleteTask::test_completed_at_is_set
PASSED  TestGetOverdueTasks::test_returns_overdue_incomplete_tasks
PASSED  TestUpdateTask::test_raises_for_missing_task

====== 18 passed in 0.12s ======
```

auto-generate tests | self-healing loop | pytest integration

5 Inline Chat — Edit Code Without Leaving the File

How to trigger Inline Chat

1. Select code in the editor
2. Press Ctrl+I (Windows/Linux)

   or Cmd+I (Mac)
3. A mini chat bar appears inline
4. Type your instruction
5. Accept / Discard the diff inline

No side panel needed — changes

Prompts to try on update_task():

"Add a try/except block and re-raise with context"
"Add a Google-style docstring"

"Add input validation for allowed_fields"

task_manager.py

```
def update_task(task_id: int, **kwargs) -> dict:
    allowed = {"title", "priority", "due_date"}
    invalid = set(kwargs) - allowed
    ...
```

Copilot: Add try/except block and re-raise with context |

Suggested changes (diff preview)

```
+     try:
+         for task in _tasks:
+             if task["id"] == task_id:
+                 ...
+         raise ValueError(f"Task {task_id} not found")
+     except (ValueError, KeyError): raise
+     except Exception as e:
+         raise RuntimeError(...) from e
```

Accept | Discard

Ctrl+I to trigger | **inline diff view** | **no side panel needed**

6 Custom Instructions — .github/copilot-instructions.md

What & Why

Copilot reads this file automatically on every chat session — no manual attachment.

Use it to encode your team's:

- Coding conventions & style rules
- Domain rules (priorities, date formats)
- Test standards & framework choices
- What to avoid (no print(), no deps)

How to create:

```
$ mkdir -p .github
$ touch .github/copilot-instructions.md
# Commit to repo — Copilot picks it up
```

.github/copilot-instructions.md

```
# GitHub Copilot Instructions — Task Manager

## Language & Style
- Python 3.11+, always use full type hints
- Google-style docstrings on every public function
- snake_case for functions, UPPER_SNAKE_CASE for constants

## Domain Rules
- Valid priorities: "low", "medium", "high" — always validate
- Dates: ISO 8601 strings (YYYY-MM-DD)

## Testing
- Framework: pytest
- Reset in-memory state with autouse fixture

## What to Avoid
- No external deps beyond stdlib + pytest
- No print() in production code
```

auto-loaded context | .github/ folder | team-wide standards

What We Covered

Auto-complete

Comment → Tab → done.
The fastest way to scaffold functions.

Ask + /explain

Chat panel for Q&A, /explain,
switching models, adding context.

Agent Mode

Multi-step autonomous coding —
reads, writes, runs code for you.

Tests + Fix

Agent generates tests, runs pytest,
and self-heals failures in a loop.

Inline Chat

Ctrl+I to edit any selection
without leaving the file.

Instructions

.github/copilot-instructions.md —
persistent, team-wide AI context.

Next step: set up Copilot in your own VS Code and try each feature on your current project →